# Introduction to Classification and Logistic Regression

# 1. What You'll Learn in This Section

In this lesson, you'll learn to:

Distinguish classification problems from regression problems by looking at the output variable.
Explain why a straight-line (linear regression) fit cannot produce usable class predictions.
Apply the sigmoid function to convert a linear equation into a probability between 0 and 1.
Build, fit, and read predictions from a logistic regression model using the standard scikit-learn workflow.

# 2. Detailed Explanation

## Classification vs. regression: two different problems

Before picking any algorithm, a machine learning engineer must first decide whether a problem is a classification problem or a regression problem. This decision comes first, because it determines which family of algorithms to use. Using a classification algorithm on a regression problem (or the reverse) gives worse results, since the algorithm was never designed for that kind of output.

The two problem types are defined by the nature of the output variable, `y`:

Regression: `y` is a continuous variable — it can take infinitely many values along a scale, such as price, marks, or salary. Predicting house prices from size, stock prices from history, or salary from employee records are all regression problems.
Classification: `y` is a discrete (class) variable — it has a fixed, finite set of possible values, such as 0/1, pass/fail, or spam/legitimate. Because the output represents a class rather than a plain number, `y` is called the class variable in classification problems.

## Real-world classification examples

Seeing a few concrete cases makes the idea of "predicting a category" click faster than definitions alone.

Spam email detection. Distinguishing spam (fake lottery wins, fraudulent loan offers, fake tax-default notices) from legitimate mail was a major problem, especially before 2010. Advances in NLP (natural language processing) and machine learning made it possible to read email text and route messages automatically. Spam classification is now considered close to 100% accurate. However, large language models are starting to make spam harder to detect by making it look more polished. Even so, machine learning still performs well because spam vocabulary stays distinctive. Modern systems

like Gmail go further, splitting legitimate mail into more than two categories: Primary (frequently-read senders), Social (social media notifications), Promotions (marketing emails), Updates (automated updates, like newspaper sign-ups), and Spam. Sorting into more than two possible classes like this is called multi-class classification.

Sentiment classification. E-commerce platforms such as Amazon or Flipkart receive too many text reviews to check manually, so companies automatically determine whether a review is positive or negative. A wave of negative reviews for a new product (a recent iPhone release, for example) is an early warning sign for sales, the same way negative day-one movie reviews reduce turnout.

Emotion classification. This works the same way as sentiment classification but recognizes more possible classes than just positive/negative.

Student pass/fail prediction. This is the running example used throughout: predicting whether a student passes or fails an exam, using hours of study as the input feature. It mirrors an earlier regression example (predicting marks from hours studied), except the target here is a class label — pass or fail — instead of a continuous score.

## Binary vs. multi-class classification

Binary classification allows exactly two class values, conventionally coded 0 and 1.

- 1 is the positive class — not "positive" in a sentiment sense, just "the thing you're interested in."
- 0 is the negative class — the thing you're not primarily interested in.
- In the pass/fail example: pass = 1 (positive class), fail = 0 (negative class).

Multi-class classification allows more than two values. Since there may be no single class of primary interest, classes are simply numbered: for example, sentiment classification with positive = class 1, neutral = class 2, negative = class 3.

Because models cannot read raw text or strings, class labels are always converted to numbers — this numeric encoding is called a symbolic representation.

## Why linear regression fails at classification

Linear regression outputs values across the entire range from negative infinity to positive infinity. In classification, only two outcomes matter (0 or 1), so a predicted value like -0.29 or 1.64 has no meaning as a class label.

Plotting the pass/fail data shows a clean pattern: students studying more than 4 hours pass (y = 1), and those studying less fail (y = 0). Hours of study is on the x-axis, and pass/fail is on the y-axis. This vertical line at "hours = 4" is called the decision boundary — the line an algorithm must learn to separate the two classes. Real-world data is rarely this cleanly separable, but this simplified dataset builds intuition before adding complexity.

Fitting a straight line through this data does not respect the 0/1 structure. For a student who studied 10 hours, the line might predict a value well above 1. For 6 hours, it might land between 0 and 1 with no probability meaning. Below the threshold, it can even predict negative numbers. None of these outputs are usable as a class decision — which motivates a function that stays pinned at the two extremes (0 or 1) except in a narrow transition zone.

# The sigmoid (logistic) function

Logistic regression is the first classification algorithm introduced. Its defining property: it always outputs a value between 0 and 1, no matter the input — exactly the range needed both for classification and for probabilities.

The core function behind logistic regression is the sigmoid function (also called the logistic function):

```
sigmoid(z) = 1 / (1 + e^(-z))
```

For any value of `z`, however large or small, `sigmoid(z)` always falls between 0 and 1. Here `z` is the same kind of linear equation used in linear regression:

```
z = w0 + w1*x1         (single feature)
z = w0 + w1*x1 + w2*x2 + w3*x3 + ...   (multiple features)
```

- `w0` is the intercept — the value of `z` when there's no information about `x` (i.e., `x = 0`).
- `w1` is the coefficient/weight for feature `x1` (here, hours of study).

Substituting `z` into the sigmoid formula gives the full logistic regression function:

```
sigmoid(z) = 1 / (1 + e^(-(w0 + w1*x1)))
```

Plotted with `z` on the x-axis and the output on the y-axis, sigmoid traces an S-shaped curve:

- As `z` approaches positive infinity, the output approaches (and flattens at) 1.
- As `z` approaches negative infinity, the output approaches (and flattens at) 0.
- At `z = 0` exactly, the output is 0.5 — the midpoint.
- Meaningful variation happens roughly between `z = -5` and `z = 5`; outside that range the curve is effectively flat at 0 or 1.

This behavior — pinned near 0 for negative inputs, pinned near 1 for positive inputs, smoothly transitioning through the middle — is exactly what classification needs, unlike the unbounded straight line linear regression produces.

# From score to decision: probability and threshold

The sigmoid output for a given input is called the classification score. Because it falls between 0 and 1, it's also read as a probability:

- `sigmoid(z)` is the probability of `y = 1` given `x`, written `P(y=1 | x)`.
- Since total probability sums to 1, `P(y=0 | x) = 1 - P(y=1 | x)`.
- If a model outputs 0.7 for a data point, that means `P(y=1|x) = 0.7` and `P(y=0|x) = 0.3`. The model always predicts the positive-class probability directly; the negative-class probability is inferred by subtraction.

> An output of 0.82 for the pass/fail model reads as "the chances of the student getting a pass grade are 82%."

Because logistic regression works in terms of how "likely" an outcome is, this style of formulation is called a likelihood formulation.

To turn a probability into a hard class decision, a threshold is applied:

| Condition | Predicted class |
| --- | --- |
| `sigmoid(z) >= 0.5` | ŷ = 1 |
| `sigmoid(z) < 0.5` | ŷ = 0 |

0.5 is the default threshold, lining up with the sigmoid's midpoint (`z = 0`). It can be adjusted by domain:

> Low-stakes domains (e.g., sentiment classification of reviews) generally keep the default 0.5.
> High-stakes domains (e.g., predicting whether a tumor is cancerous) may use a higher threshold such as 0.6 or 0.7, so the model only predicts the positive class when more confident.

The point `z = 0` (equivalently, probability = 0.5) is the model's decision boundary — in the pass/fail example, the vertical line at 4 hours of study. Values of `z` above 0 push the prediction toward the positive class; values below 0 push toward the negative class.

This kind of probability-based thinking has an everyday parallel: deciding whether it will rain today, or whether a class session will happen (which depends on circumstances like illness or family emergencies). In both cases, people reason in terms of "what is the probability of X" before reaching a yes/no decision — the same reasoning logistic regression formalizes mathematically.

Watch out for: it's tempting to treat the classification score as the final answer, but it's only a probability until a threshold is applied — the threshold is what actually produces the class label.

## Logistic regression as a linear classifier

Classifiers fall into two broad types:

> Linear classifiers — most classification algorithms, including logistic regression, belong here. Even though the sigmoid curve is S-shaped, its decision-relevant behavior — constant near 0, constant near 1, with a roughly straight increasing segment in the middle — keeps it in the "linear" family. Scikit-learn accordingly places `LogisticRegression` in the same `linear_model` module as `LinearRegression`.
> Non-linear classifiers — neural networks are the example given. They can learn arbitrarily complex, non-linear decision boundaries (irregular shapes separating classes scattered in complicated patterns), which linear classifiers cannot represent.

# Manual worked example: computing sigmoid by hand

Before calling any library function, it helps to compute sigmoid by hand. Assume fixed (hypothetical, not yet learned) weight values `w0 = -6` and `w1 = 1.5`:

```
z = w0 + w1 * hours = -6 + 1.5 * hours
```

| Hours of study | z = -6 + 1.5×hours | sigmoid(z) (probability of pass) | Predicted class |
|---|---|---|---|
| 1 | -4.5 | ≈ 0.011 | fail |
| 2 | -3.0 | ≈ 0.047 | fail |
| 4 | 0.0 | 0.5 | pass (boundary) |
| 5 | 1.5 | ≈ 0.82 | pass |
| 6 | 3.0 | ≈ 0.95 | pass |

At exactly 4 hours, `z = 0` and sigmoid outputs exactly 0.5 — a genuine 50/50 decision point, and where the decision boundary sits. Below 4 hours, the probability of passing is very low; above 4 hours it climbs quickly toward 1. Whenever the probability is greater than or equal to 0.5, the point is classified as the positive class (pass) — every point must resolve to one class or the other, never left undecided.

# Code walkthrough: building and fitting a logistic regression model

The dataset used here is a small "toy" dataset, small enough to visualize on screen. It's built to make the algorithm's behavior easy to see before working with the thousands-to-lakhs of real-world data points typical in practice.

Setup fixes the plot size and random seed for reproducibility:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams['figure.figsize'] = (7, 4.5)
np.random.seed(42)
```

The toy dataset has seven students — three who studied fewer than 4 hours and failed, four who studied 4 or more hours and passed:

```
demo = pd.DataFrame({
    'hours': [1, 2, 3, 4, 5, 6, 7],
    'pass':  [0, 0, 0, 1, 1, 1, 1]
})
```

Fitting a plain `LinearRegression` on this data demonstrates why it's unsuitable:

```
from sklearn.linear_model import LinearRegression

lin_model = LinearRegression()
lin_model.fit(demo[['hours']], demo['pass'])

hours_range = np.linspace(0, 9, 100).reshape(-1, 1)
lin_predictions = lin_model.predict(hours_range)
```

`reshape(-1, 1)` converts the generated values into a NumPy array shaped rows-by-columns (one feature column, one row per data point)—the format scikit-learn expects. At the extremes, this fitted linear model predicts about -0.29 at 0 hours and about 1.64 at 9 hours. Both values fall outside the meaningful 0–1 range, confirming a straight-line fit isn't usable here.

Implementing sigmoid manually and testing it across a wide range shows the S-curve directly:

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z_values = np.linspace(-8, 8, 200)
p_values = sigmoid(z_values)
```

Plotting `z_values` against `p_values` produces the S-shaped curve: flat near 0 for very negative `z`, flat near 1 for very positive `z`, and a smooth transition around `z = 0`.

Fitting an actual `LogisticRegression` on the same data:

```
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression()
clf.fit(demo[['hours']], demo['pass'])

print(clf.intercept_[0])    # -3.99
print(clf.coef_[0][0])      # 1.15
```

After fitting, scikit-learn stores the learned intercept and coefficient as attributes ending in an underscore (`intercept_`, `coef_`) — its convention for values only available after `fit()`. For this dataset, the fitted model produced an intercept of -3.99 and a coefficient of 1.15 for `hours`.

Getting probabilities and class predictions:

```
demo['predicted_proba'] = clf.predict_proba(demo[['hours']])[:, 1]
demo['predicted_class'] = clf.predict(demo[['hours']])
```

Sample results: at 1 hour, predicted probability ≈ 0.05 (well below 0.5) gives predicted class 0 (fail). At 4 hours, predicted probability ≈ 0.644 (above 0.5) gives predicted class 1 (pass), matching the actual label. `predict_proba` returns two columns per row—the negative-class probability (column 0) and the positive-class probability (column 1). Only the positive-class column is normally used directly, since the negative-class value is inferred by subtracting from 1.

A second, purely manual pass (`demo_work`) reruns the same fixed weights (`w0 = -6`, `w1 = 1.5`) without calling `LogisticRegression` at all:

```
demo_work = pd.DataFrame({'hours': [1, 2, 4, 5, 6]})

w0, w1 = -6, 1.5
demo_work['z'] = w0 + w1 * demo_work['hours']
demo_work['p'] = sigmoid(demo_work['z'])

demo_work['prediction'] = np.where(demo_work['p'] >= 0.5, 'pass', 'fail')
```

`np.where` applies the 0.5 threshold condition across the whole column at once. This reproduces the same z, probability, and pass/fail values from the earlier hand-computed table. It confirms that `LogisticRegression.fit()` performs the same sigmoid-based computation, except the fitting process discovers the optimal weights automatically (`w0 = -3.99`, `w1 = 1.15` here) instead of them being supplied by hand.

Finally, generating 200 evenly spaced test values across the 0–9 hour range and passing them through `predict_proba` produces a smooth predicted-probability curve:

```
hours_test = np.linspace(0, 9, 200).reshape(-1, 1)
prob_curve = clf.predict_proba(hours_test)[:, 1]
```

Plotting the training points together with this curve and a horizontal reference line at 0.5 shows the S-curve passing correctly through the training points. It stays near 0 for failing students, near 1 for passing students, and crosses 0.5 right around the 4-hour mark—the model's learned decision boundary. Raising or lowering this threshold line (e.g., to 0.8) changes which probability values get classified as the positive class.

## The common machine learning workflow pattern

Across virtually every scikit-learn classification (and regression) algorithm — from logistic regression through to deep neural networks — the same four-step pattern recurs:

```
Create algorithm object

e.g. LogisticRegressionCall fit on training dataCall predict_proba

for probability scoreCall predict
```

`for final class label`

1. Create an object of the algorithm's class (e.g., `LogisticRegression()`) — sometimes called constructing or instantiating the class.
2. Call `fit()` on the object with the training data — this trains the model and produces the optimal weight values (intercept and coefficients).
3. Call `predict()` to get the final predicted class label for new data.
4. Call `predict_proba()` to get the underlying probability/classification score before thresholding.

This pattern is described as one of the most important, reusable pieces of code to write repeatedly, regardless of which specific algorithm is being trained.

## Evaluation metrics, true/false labels, and why logistic regression matters going forward

Classification problems use different evaluation metrics than regression. Accuracy score and confusion matrix are the relevant metrics for classification, common across algorithms from traditional machine learning up through transformers. Detailed discussion is left for a dedicated future session; for now, treat them simply as functions that judge how good a classification model's performance is.

A common question: do true/false labels count as classification? Yes — true/false is just another human-readable interpretation of the same underlying 1/0 (or positive/negative) numeric class values. The value actually stored and processed by the model is always numeric, since models cannot work with raw strings directly; "true/false" and "positive/negative" are interpretation conventions closer to human language.

Logistic regression is the first of several classification algorithms; decision trees and k-nearest neighbors (KNN) follow in upcoming coverage. These are fundamental algorithms that remain valid and competitive — not outdated — because in many practical scenarios they outperform more modern approaches such as transformers. The sigmoid function itself is bigger than logistic regression alone. The same S-shaped, 0-to-1-bounded function reappears in neural networks, deep neural networks, and transformers. A solid grasp of its shape — flattening at 0 and 1, with a 0.5 midpoint at `z = 0` — pays off throughout later material. Learners without a strong mathematics background benefit from extra self-study time on the underlying linear-equation and sigmoid concepts. Focus on these specific pieces—not full linear algebra, differential equations, or integration—since machine learning is built directly on top of mathematics. Many more advanced algorithms build on concepts introduced through logistic regression, including neural networks, deep neural networks, and support vector machines (SVM), which is why it forms the base for further classification topics.

# 3. Key Takeaways

Classification predicts a discrete class label (like pass/fail or spam/legitimate); regression predicts a continuous number — deciding which one you have comes before choosing an algorithm.
Linear regression produces unbounded output and can't respect a 0/1 class structure, which is why the sigmoid function — always bounded between 0 and 1 — is used instead.
The sigmoid function turns a linear equation `z = w0 + w1*x1` into a probability; applying a threshold (default 0.5) converts that probability into a final class prediction.

The point where `z = 0` and probability = 0.5 is the decision boundary — in the pass/fail example, this sits at 4 hours of study.
The four-step workflow — create the object, `fit()`, `predict_proba()`, `predict()` — recurs across nearly every scikit-learn algorithm, making it worth memorizing early.

Mental model: Think of logistic regression as a dimmer switch built from a linear equation. Instead of jumping straight from "off" to "on," it slides smoothly from 0 to 1 through the sigmoid curve. The 0.5 threshold is where you decide to finally flip the switch to a firm class decision.